

Using Primitive Data Types

Objective

This lab will begin to introduce the Java syntax to the new Java programmer. You will create your first Java program and experiment with using different primitive data types.

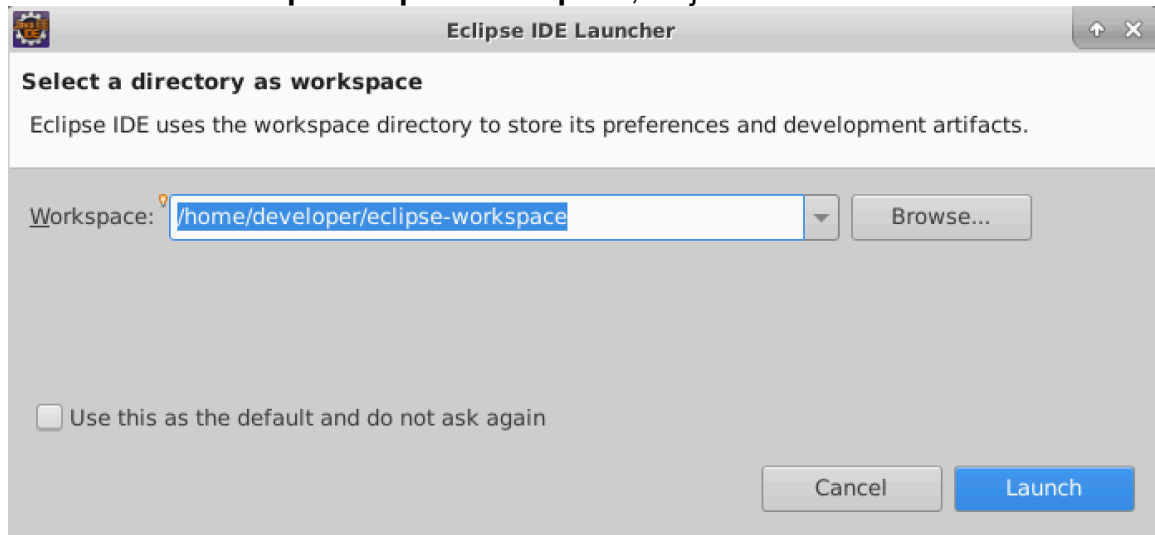
Overview

In this lab you will create a new Java project and your first Java class. While the details of object-oriented programming and Java class structure are yet to come, this initial Java class will allow you to begin writing some basic Java code.

Step by Step Instructions

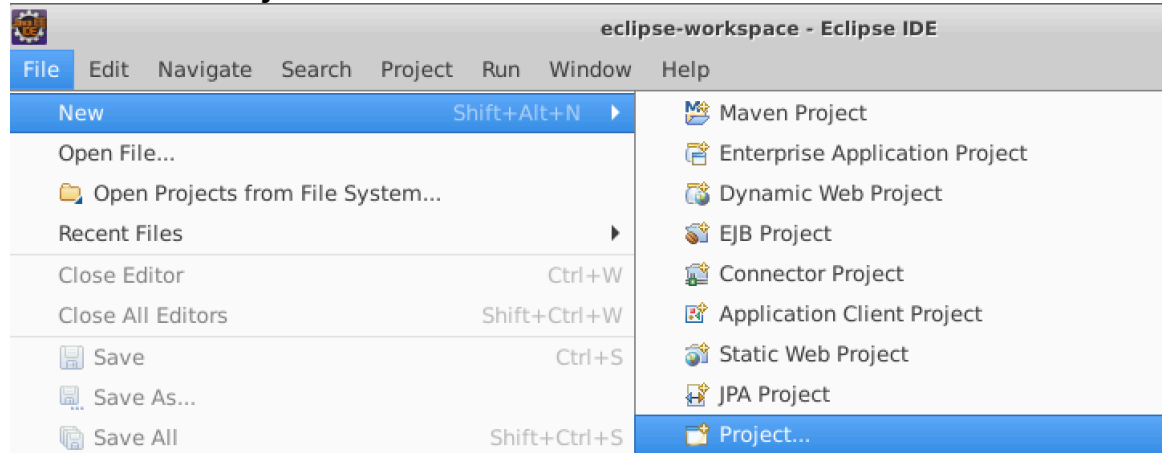
Create a Workspace, Project, and Package

1. Launch your IDE and use the default workspace for your lab exercises. The default workspace location should be left unchanged in this environment and it is: **/home/developer/eclipse-workspace**, so just click the **Launch** button.

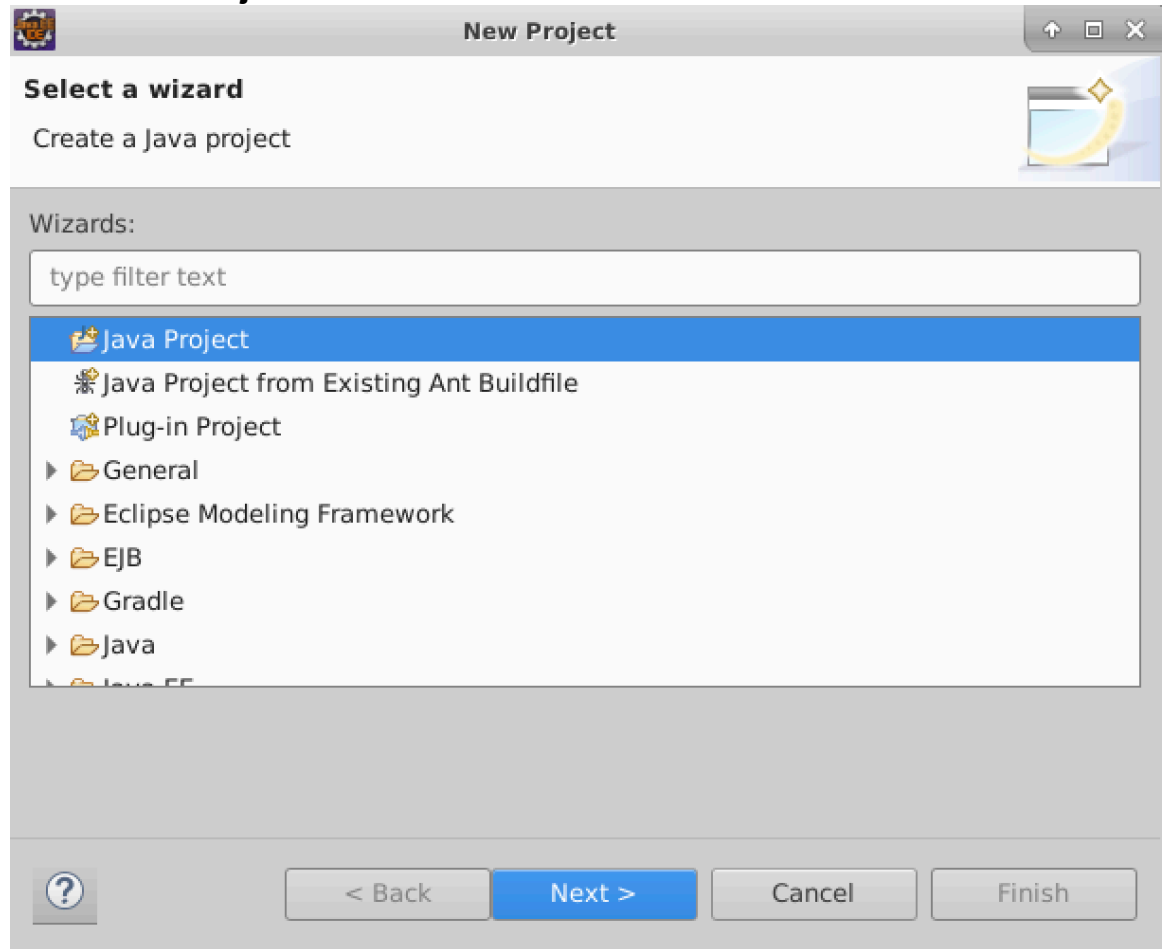


2. Create a new Java project for the first few exercises. Name the project **ClassExercises**.

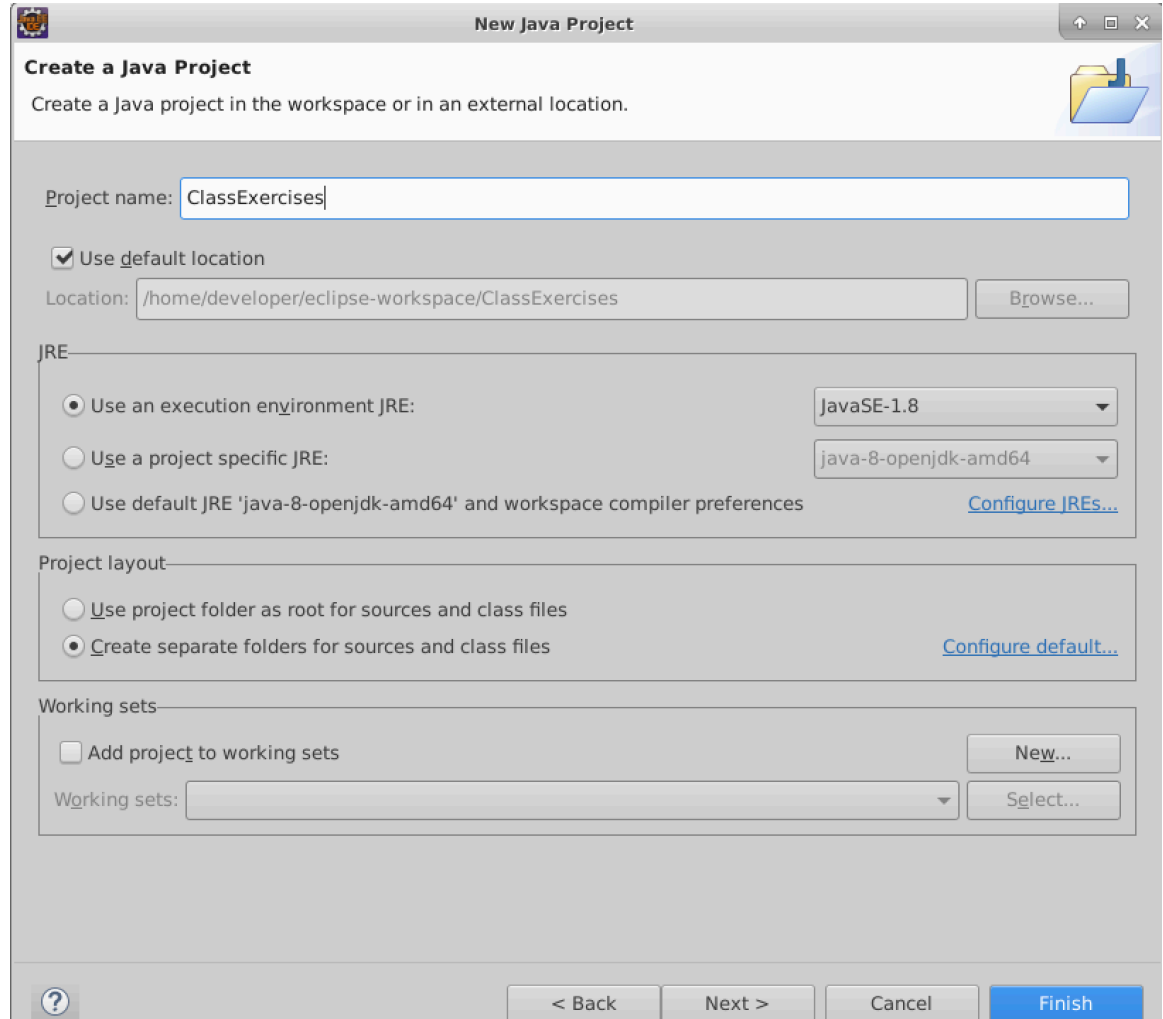
- a. Select **File -> Project...**



- b. Select **Java Project**

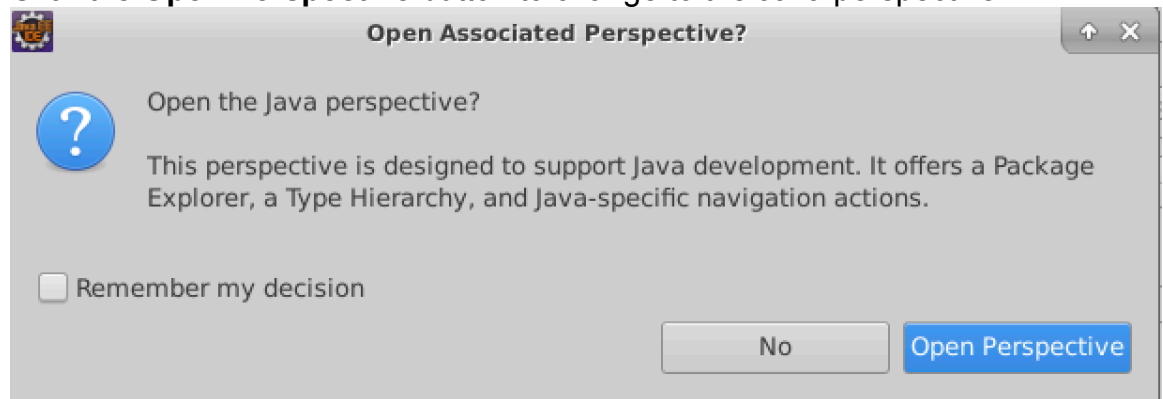


- c. Enter **ClassExercises** as the Project Name and click the **Finish** button.



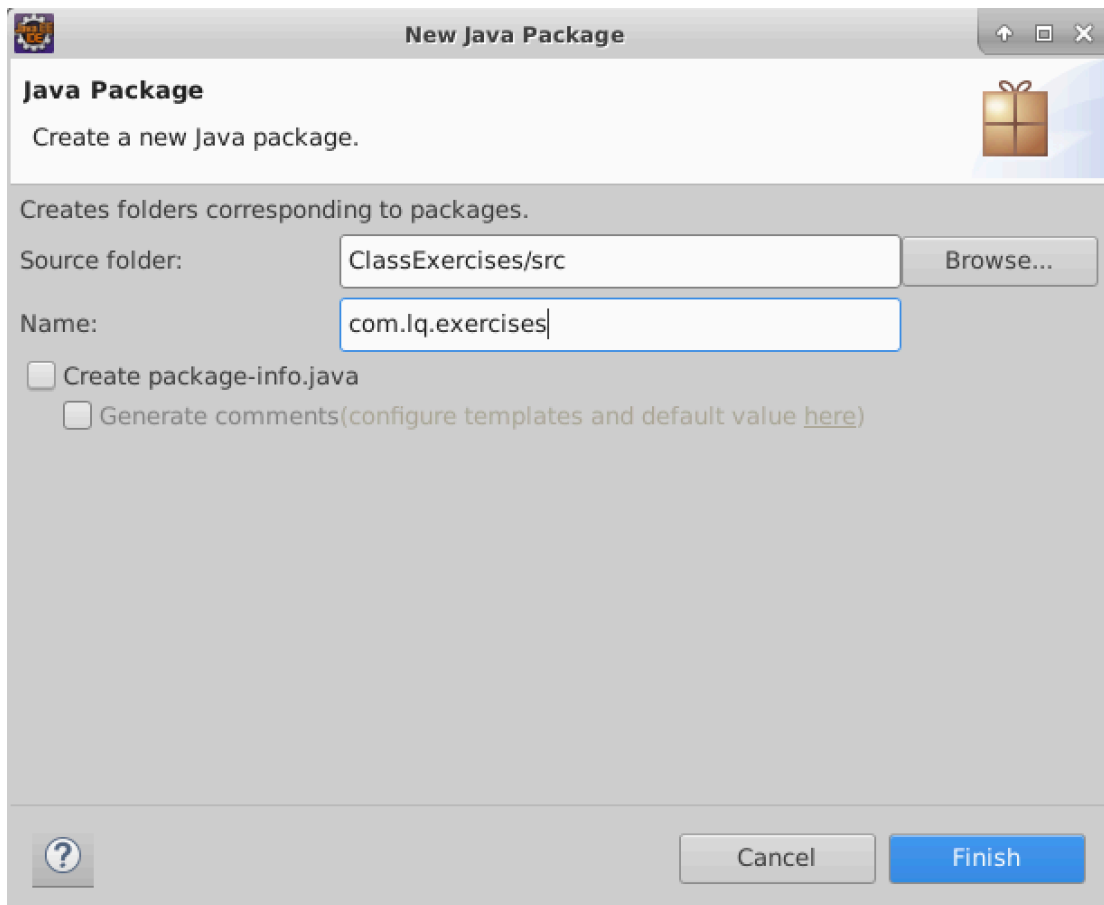
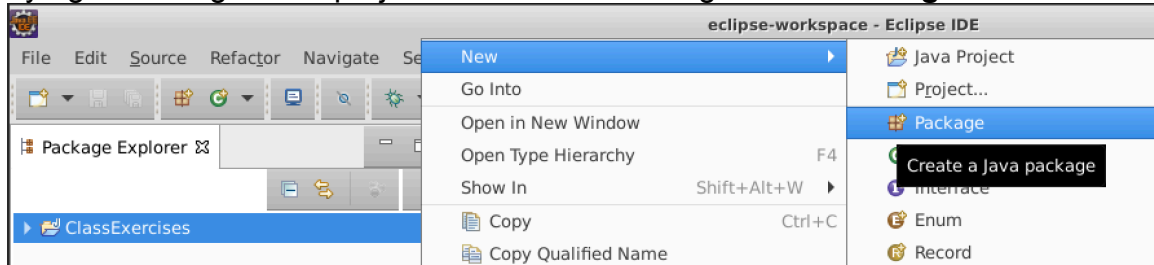
The screenshot shows the 'New Java Project' dialog box. The title bar says 'New Java Project'. The main heading is 'Create a Java Project' with a subtext 'Create a Java project in the workspace or in an external location.' and a folder icon. The 'Project name' field contains 'ClassExercises'. The 'Use default location' checkbox is checked, and the 'Location' field shows '/home/developer/eclipse-workspace/ClassExercises' with a 'Browse...' button. The 'JRE' section has three radio buttons: 'Use an execution environment JRE:' (selected), 'Use a project specific JRE:', and 'Use default JRE 'java-8-openjdk-amd64' and workspace compiler preferences'. The first option has a dropdown menu showing 'JavaSE-1.8'. The second option has a dropdown menu showing 'java-8-openjdk-amd64'. There is a 'Configure JREs...' link. The 'Project layout' section has two radio buttons: 'Use project folder as root for sources and class files' and 'Create separate folders for sources and class files' (selected). There is a 'Configure default...' link. The 'Working sets' section has an 'Add project to working sets' checkbox and a 'Working sets:' dropdown menu. There are 'New...' and 'Select...' buttons. At the bottom, there are buttons for '< Back', 'Next >', 'Cancel', and 'Finish'.

- d. Click the **Open Perspective** button to change to the Java perspective.

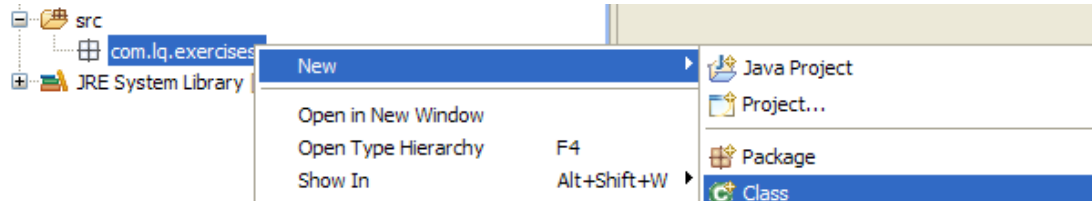


The screenshot shows the 'Open Associated Perspective?' dialog box. The title bar says 'Open Associated Perspective?'. The main heading is 'Open the Java perspective?' with a question mark icon. The text below says 'This perspective is designed to support Java development. It offers a Package Explorer, a Type Hierarchy, and Java-specific navigation actions.' There is a 'Remember my decision' checkbox. At the bottom, there are buttons for 'No' and 'Open Perspective'.

3. Create a package named **com.lq.exercises**. This can be done in most IDEs by right-clicking on the project name and selecting **New > Package**.



4. Create a new class named **Lab2** in the **com.lq.exercises** package. This can be done in most IDEs by right-clicking on the package and selecting **New > Class**.



The new class wizard will appear. Enter the name of the class, check the box for a main method, and check the generate comments method. Click the **Finish** button.

A screenshot of the 'New Java Class' dialog box. The title bar says 'New Java Class'. The main heading is 'Java Class' with a subtitle 'Create a new java class.' and a green 'C' icon. The 'Source folder' is 'ClassExercises/src' with a 'Browse...' button. The 'Package' is 'com.lq.exercises' with a 'Browse...' button. There is an unchecked checkbox for 'Enclosing type' with a 'Browse...' button. The 'Name' field contains 'Lab2' and is circled in red. The 'Modifiers' section has radio buttons for 'public' (selected), 'package', 'private', and 'protected', and checkboxes for 'abstract', 'final', and 'static'. The 'Superclass' field contains 'java.lang.Object' with a 'Browse...' button. The 'Interfaces' section is empty with an 'Add...' button and a 'Remove' button. Under 'Which method stubs would you like to create?', the checkbox for 'public static void main(String[] args)' is checked and circled in red, along with 'Inherited abstract methods'. The text 'Do you want to add comments? (Configure templates and default value [here](#))' is followed by a checked checkbox for 'Generate comments', which is also circled in red. At the bottom are 'Cancel' and 'Finish' buttons.

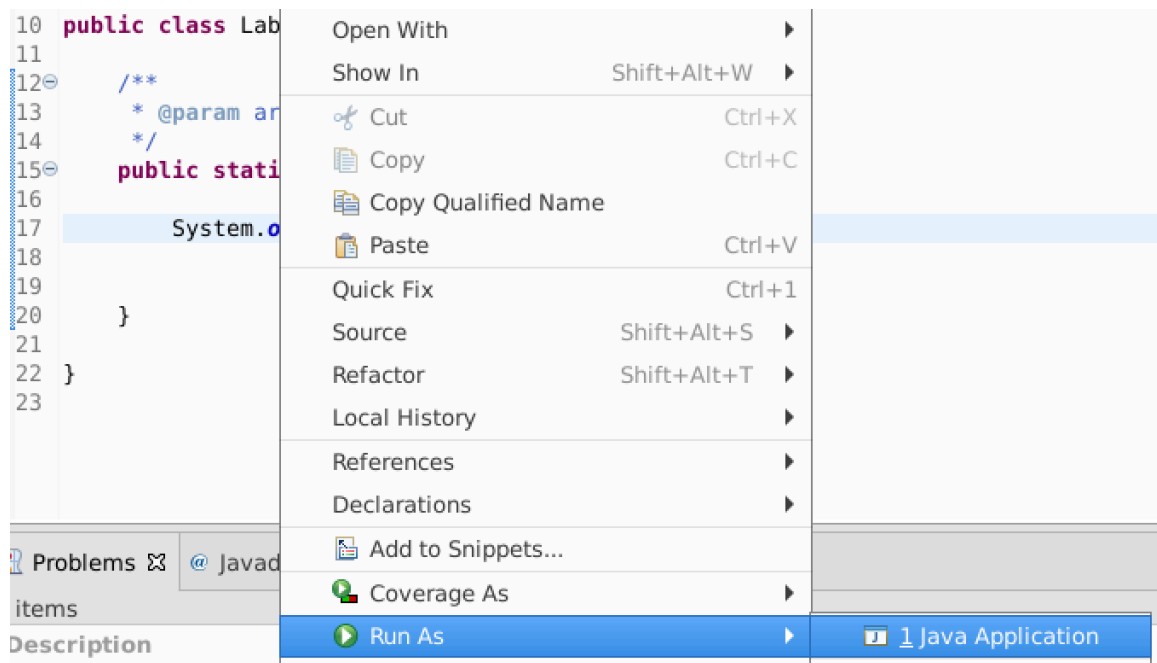
5. The **Lab2** class will open in the editor. It will contain a beginning curly brace '{' and an ending curly brace '}' for the class definition. Proceed with the lab exercises that follow.

Implement the main() method

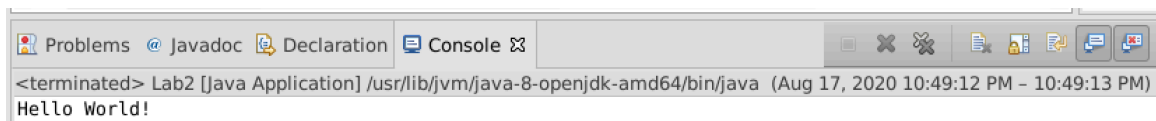
6. Remove the TODO comment. It is there only to remind you that the method was automatically created for you and that you must implement what the method is to do.
7. Add a print statement to your `main()` method that says, "**Hello World!**". We will cover the details of how this all works later but for now it will look like the following:

```
1+ /**  
4 package com.lq.exercises;  
5  
6- /**  
7  * @author developer  
8  *  
9  */  
10 public class Lab2 {  
11  
12-     /**  
13         * @param args  
14         */  
15-     public static void main(String[] args) {  
16  
17         System.out.println("Hello World!");  
18  
19  
20     }  
21  
22 }  
23
```

8. Save your changes and execute your small Java program. In most IDEs this can be accomplished by right-clicking on the class name and selecting **Run As > Java Application** or by clicking the run button on the toolbar.



9. View the results in the console view. It should be similar to the following:



Define Primitive Data Types

10. At the beginning of the main() method, declare the following variables with the specified characteristics:
- An int with the name **width** and no initial value.

- b. An int with the name **height** and no initial value.
- c. An int with the name **area** and no initial value.
- d. A double with the name **radius** and an initial value of **10.0**.
- e. A double with the name **pi** and an initial value of **3.14**.
- f. A boolean with the name **result** and an initial value of **true**.

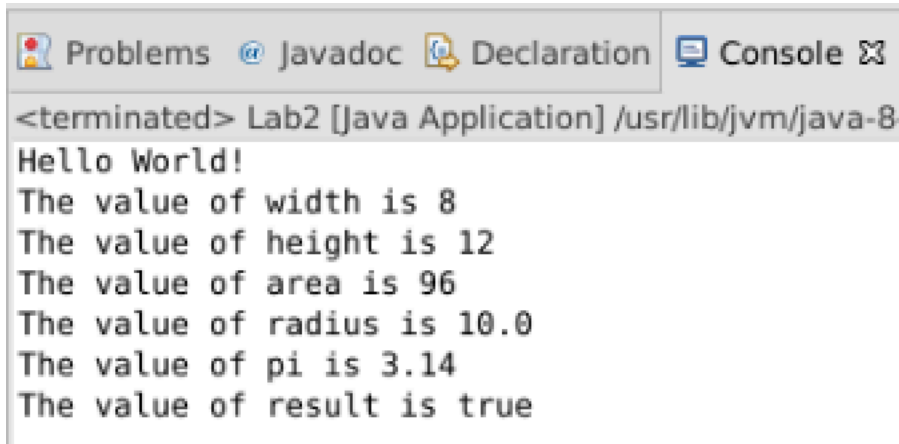
In the body of the main() method, perform the following variable assignments:

- g. Assign a value of **8** to **width**.
- h. Assign a value of **12** to **height**.
- i. Assign a value of **96** to **area**.

11. Near the bottom of the main() method, print the values of each variable using `System.out.println()` . Use the following as a guide:

```
System.out.println("The value of width is " + width);
```

12. Execute your program. Your output should be similar to the following:



```
<terminated> Lab2 [Java Application] /usr/lib/jvm/java-8.  
Hello World!  
The value of width is 8  
The value of height is 12  
The value of area is 96  
The value of radius is 10.0  
The value of pi is 3.14  
The value of result is true
```

Define Arrays

13. Near the top of your main() method, define the following arrays:

- j. An array of 12 **ints** named **daysInMonths**.
- k. An array of 12 **String** references named **monthNames** – initialize this array at the time it is declared with the names of the 12 months (refer to your course book for the syntax).

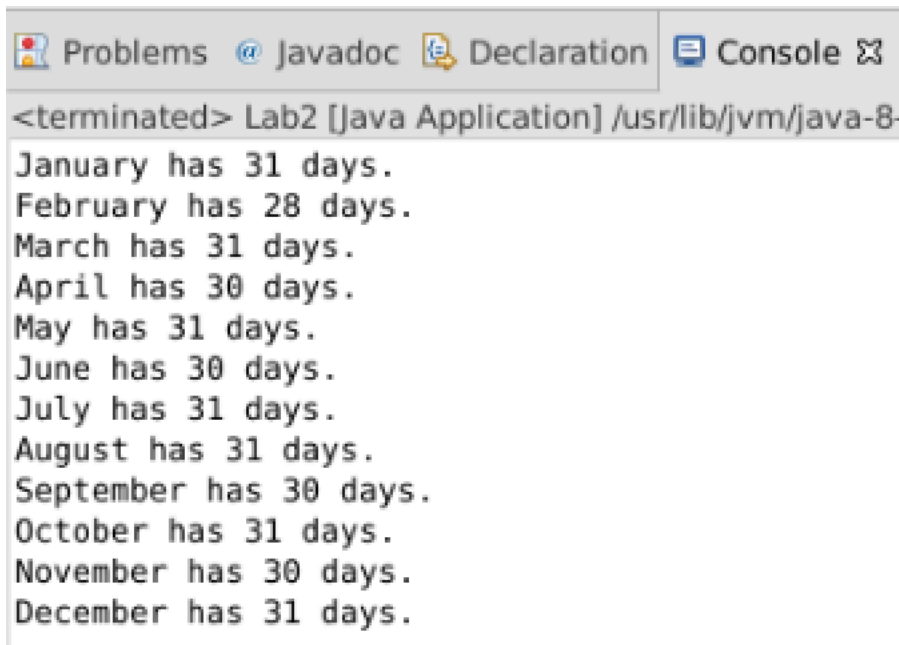
14. Write 12 lines of code to assign the number of days in each month to the `daysInMonths[]` array elements (do not worry about leap year!). See output in step 16 below for days in months.

15. Write a print statement for each month that will display the name of the month and the number of days it contains. Use the following as a guide:

```
System.out.println( monthNames[0] + " has " + daysInMonths[0] + " days." );
```

If you are already familiar with loops, you may use them. If not, then use 12 output statements.

16. Execute your program. Your output should be similar to the following:



The screenshot shows an IDE window with four tabs: Problems, Javadoc, Declaration, and Console. The Console tab is active, displaying the output of a Java application. The output consists of 12 lines, each representing a month and its number of days. The text is as follows:

```
<terminated> Lab2 [Java Application] /usr/lib/jvm/java-8-  
January has 31 days.  
February has 28 days.  
March has 31 days.  
April has 30 days.  
May has 31 days.  
June has 30 days.  
July has 31 days.  
August has 31 days.  
September has 30 days.  
October has 31 days.  
November has 30 days.  
December has 31 days.
```